

DM548: Computer Architecture & System Programming DM588: Computer Architecture

Fall 2026

A Top-Level View and Interconnection Structures

- Instruction Cycle
- Interrupts
- I/O Functionality
- Interconnection Structures
 - Bus Interconnection
 - QPI
 - PCIe

Where are we?

Recall:

- ▶ **Architecture**: what can it do, as seen by the user.
E.g., instruction set.
- ▶ **Organisation**: the implementation of an architecture.
E.g., hardware units, interconnections, memory technology.

Today: an overview of

- ▶ internal computer organisation,
- ▶ how a computer executes,
- ▶ how components communicate.

Organisation

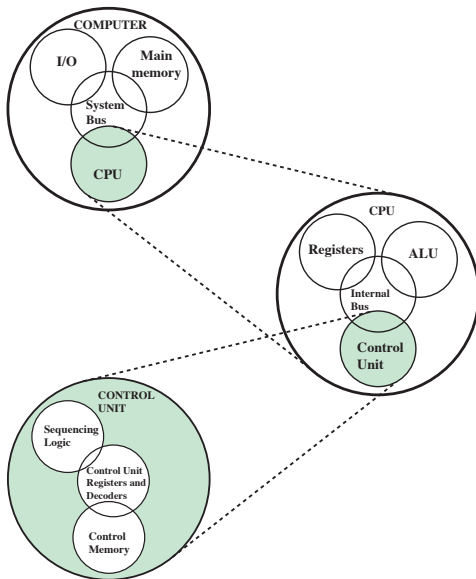


Figure 1.1 A Top-Down View of a Computer

Computer Components

- ▶ Modern computer design is based on concepts developed by John von Neumann:
 - ▶ Data and instructions are stored in a single read-write memory.
 - ▶ The memory is addressable by location, without regard to the type of data contained there.
 - ▶ Execution occurs in a sequential fashion, from one instruction to the next, unless explicitly modified.

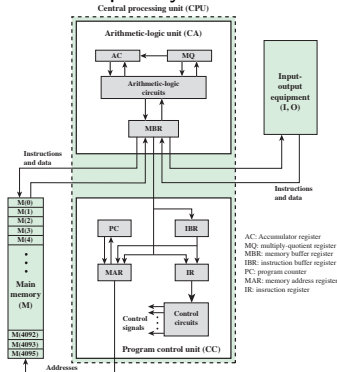
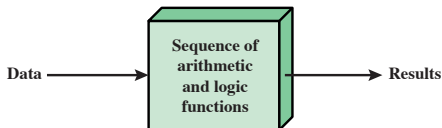


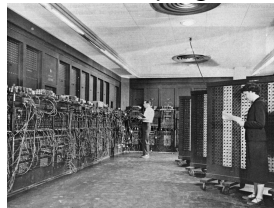
Figure 1.5 IAS Structure

General-Purpose vs. Hardwired

- **Hardwired:** the configuration of the machine itself is the “program”.

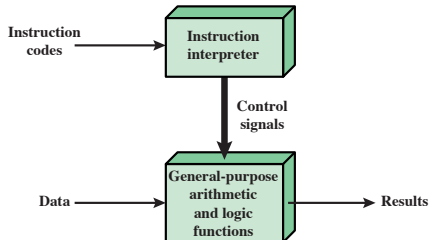


(a) Programming in hardware



The ENIAC.

- **General-purpose:** instructions are decoded into a specific configuration of control signals.

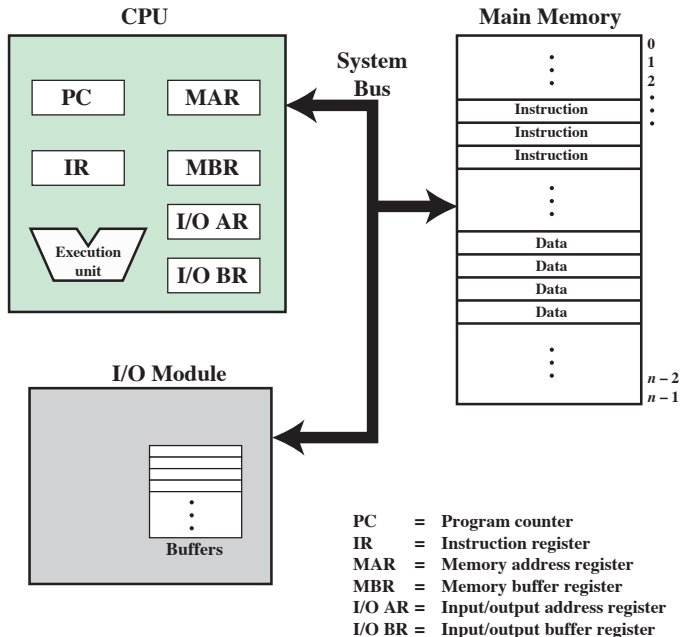


A Top-Level View

- ▶ Software
 - ▶ A sequence of codes, i.e., instructions.
 - ▶ Interpreted by hardware.
 - ▶ Much easier to change than rewiring hardware.
- ▶ Main computer components
 - ▶ CPU
 - ▶ Instruction interpreter
 - ▶ Module of general-purpose arithmetic and logic functions
 - ▶ I/O components
 - ▶ Input: accept data and instructions, and convert them to an internal representation.
 - ▶ Output: a means of reporting results.
 - ▶ Main memory: temporary storage for instruction and data.

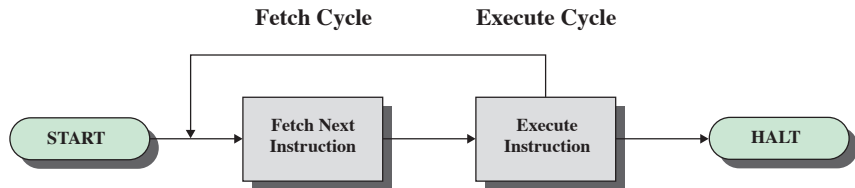
Note: in this simplified view, components like a hard drive, SSD, or a graphics card are I/O components.

A Minimalistic Computer



- Instruction Cycle
- Interrupts
- I/O Functionality
- Interconnection Structures
 - Bus Interconnection
 - QPI
 - PCIe

How Does Our Simple Machine Work



- ▶ The processing required for a single instruction is called an **instruction cycle**.
- ▶ In this simple form, it consists of:
 1. Fetch cycle
 2. Execute cycle
- ▶ Instruction examples:
 - ▶ Load from memory to an accumulator register (AC).
 - ▶ Store AC in memory.
 - ▶ Add to AC from memory.

The Instruction Cycle

Fetch:

- ▶ At the beginning of each instruction cycle the CPU fetches an instruction from memory.
- ▶ The program counter (PC) holds the address to fetch from.
- ▶ The fetched instruction is loaded into the instruction register (IR).

Execute:

- ▶ The CPU interprets the instruction and performs the specified action.
- ▶ Unless otherwise specified, the PC is incremented.

The Instruction Cycle

Fetch:

- ▶ At the beginning of each instruction cycle the CPU fetches an instruction from memory.
- ▶ The program counter (PC) holds the address to fetch from.
- ▶ The fetched instruction is loaded into the instruction register (IR).

Execute:

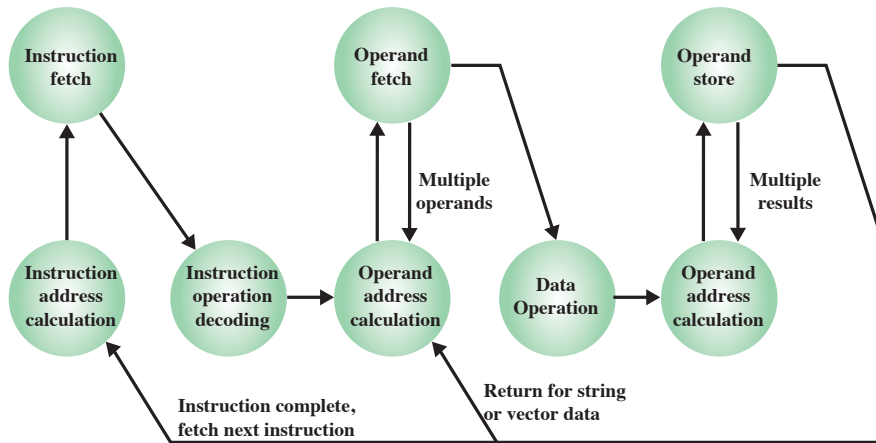
- ▶ The CPU interprets the instruction and performs the specified action.
- ▶ Unless otherwise specified, the PC is incremented.

Is that it? Probably not:

- ▶ Often an instruction would have operands.
- ▶ We may use other memory addressing schemes than direct.
- ▶ We need to do I/O operations as well.

The instruction cycle must be more complicated.

More Advanced Instruction Cycle



- Instruction Cycle
- Interrupts
- I/O Functionality
- Interconnection Structures
 - Bus Interconnection
 - QPI
 - PCIe

Interrupts

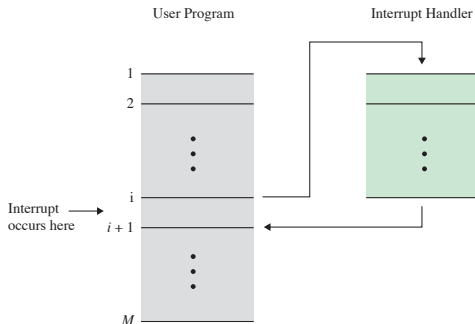
- ▶ Main memory and I/O modules may have their own specialized processors.
- ▶ What if they want to tell the CPU something? E.g.,
 - ▶ The network controller have incoming data.
 - ▶ You pressed a key, the CPU should do something.
- ▶ **Polling**: the CPU constantly checks the state of things.
 - ▶ May be inefficient if the events are rare.
 - ▶ If the event quickly changes, the CPU may miss it.
- ▶ **Interrupts**: modules can notify the CPU and interrupt it.
- ▶ Virtually all computers provide interrupt mechanisms.

We will get back to interrupts in a later lecture on I/O and processor structure.

Types of Interrupts

- ▶ **Program:** as a result or direct side-effect of instruction execution.
 - ▶ Arithmetic overflow, division by zero, ...
 - ▶ Attempt to execute an illegal machine instruction.
 - ▶ Attempt to references memory outside the allowed user space.
- ▶ **Timer:** generated by a timer within the processor.
 - ▶ Can be used by the OS to perform certain functions on a regular basis.
 - ▶ For example, switch the running program.
- ▶ **I/O:** generated by an I/O controller.
 - ▶ Signal normal completion of an operation.
 - ▶ Signal various error conditions.
- ▶ **Hardware failure:** generated by hardware component.
 - ▶ Power failure.
 - ▶ Memory parity error.

How Does an Interrupt Work?



1. An interrupt is signalled.
2. The processor finishes the current instruction.
3. The processor jumps to a predefined piece of code, called an **interrupt handler** or **interrupt service routine**.
4. Afterwards it jumps back to where it was before.

(See next slides for details)

Each type of interrupt may have a different handler.

Program Flow With and Without Interrupts

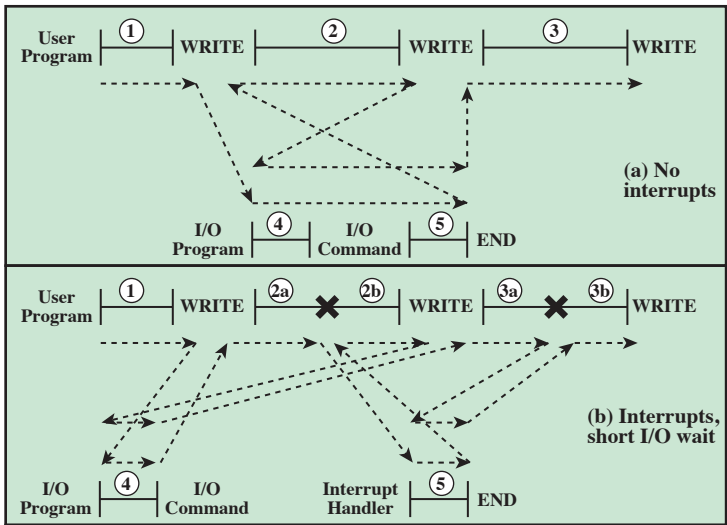


Fig. 3.7 (note, the figure in the book seems to have a bug in the third scenario (not shown here))

Program Timing With and Without Interrupts

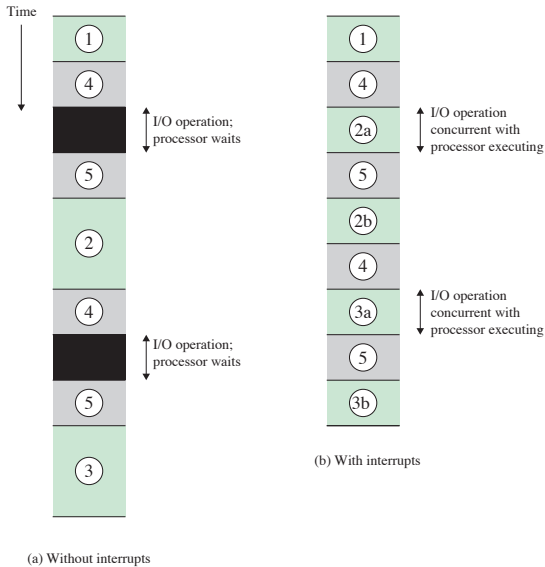


Figure 3.10 Program Timing: Short I/O Wait

Program Timing With and Without Interrupts

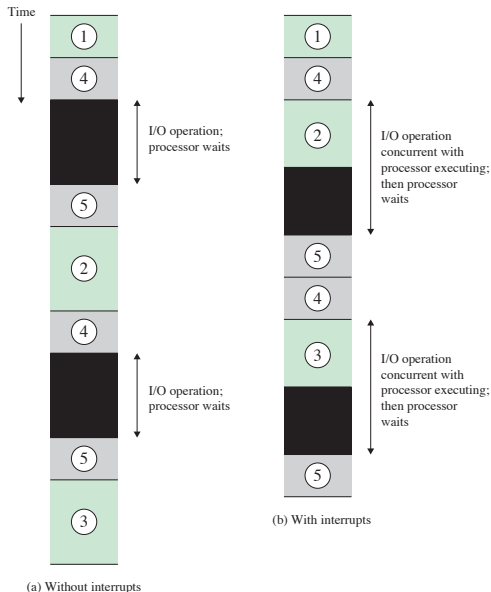
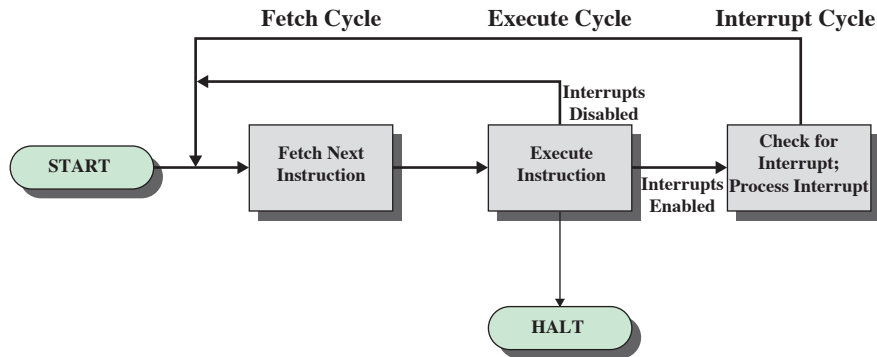


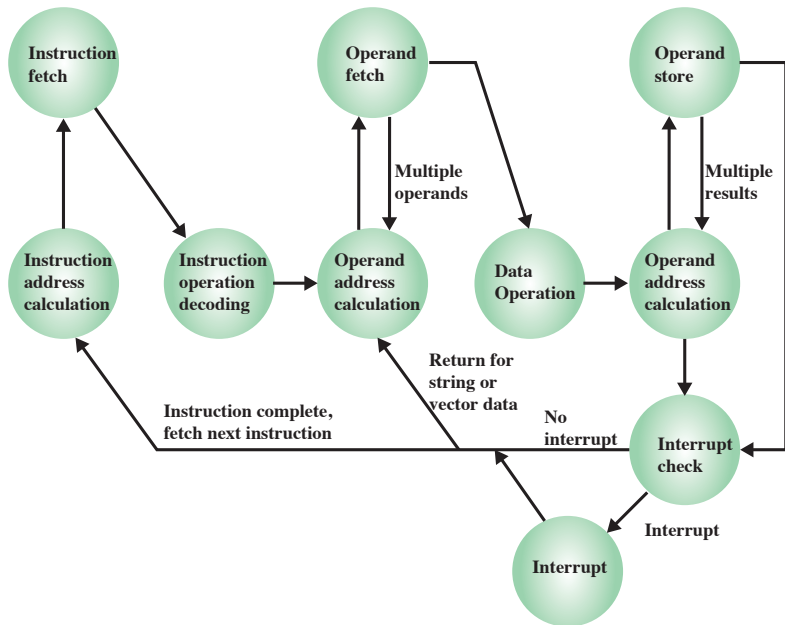
Figure 3.11 Program Timing: Long I/O Wait

Simple Instruction Cycle With Interrupts

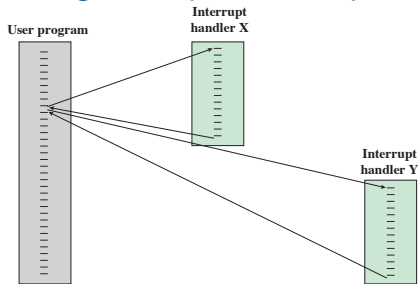


- ▶ Disabling interrupts is a very powerful operation.
- ▶ It basically means that we get complete control.
- ▶ Usually, only the operating system kernel is allowed to do that.
(In x86 see the `cli` instruction)

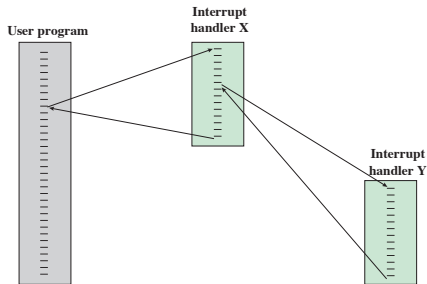
Instruction Cycle State Diagram With Interrupts



Two Ways of Handling Multiple Interrupts



(a) Sequential interrupt processing



(b) Nested interrupt processing

Handling Multiple Interrupts

Sequential:

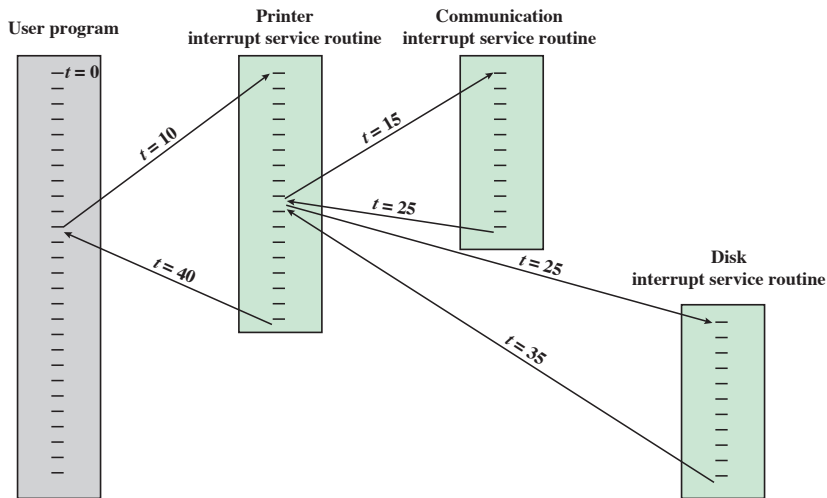
- ▶ Only handle 1 interrupt at a time.
- ▶ That is, the first thing each interrupt handler does is to disable interrupts.
- ▶ Generally, if an interrupt is signalled, it will remain pending until noticed.
- ▶ When the handler returns it enables interrupts again.
- ▶ This scheme does not consider that some interrupts may be time critical.
- ▶ For example, an interrupt from a printer is not really critical.
- ▶ But those from a high-speed network controller are.
Buffers may overflow, data may get lost.

Nested:

- ▶ An interrupt handler may it self be interrupted.
- ▶ Also a rather extreme scheme.
- ▶ E.g., a printer should probably not be allowed to interrupt other interrupts.

Prioritized Interrupt Handling

- ▶ The best of both worlds.
- ▶ Each interrupt has a priority.
- ▶ Higher priority interrupts may interrupt lower priority handlers.



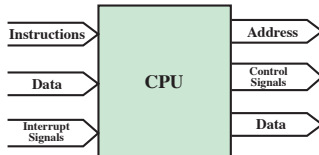
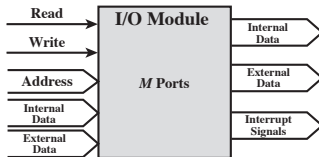
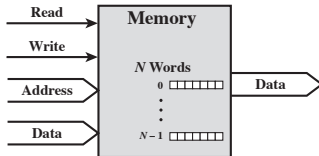
- Instruction Cycle
- Interrupts
- I/O Functionality
- Interconnection Structures
 - Bus Interconnection
 - QPI
 - PCIe

I/O Functionality

- ▶ I/O modules can exchange data directly with the CPU.
- ▶ This communication is done with special I/O instructions, instead of referencing memory.
- ▶ However, it is often desirable to let I/O modules read/write directly from main memory.
 - ▶ The CPU gives the module authority to read/write to/from main memory.
 - ▶ The I/O module then issues commands to main memory without tying up the CPU.
 - ▶ E.g., let a network controller read data to be sent while the CPU does something else.
 - ▶ This scheme is called Direct Memory Access (DMA).

- Instruction Cycle
- Interrupts
- I/O Functionality
- Interconnection Structures
 - Bus Interconnection
 - QPI
 - PCIe

The Computer as a Set of Components



What Needs to be Interchanged?

- ▶ **Memory to CPU**: an instruction or unit of data.
- ▶ **CPU to memory**: a unit of data.
- ▶ **I/O to CPU**: read data from I/O device through I/O module.
- ▶ **CPU to I/O**: send data to I/O device through I/O module.
- ▶ **I/O to/from memory**: read/write using Direct Memory Access (DMA).

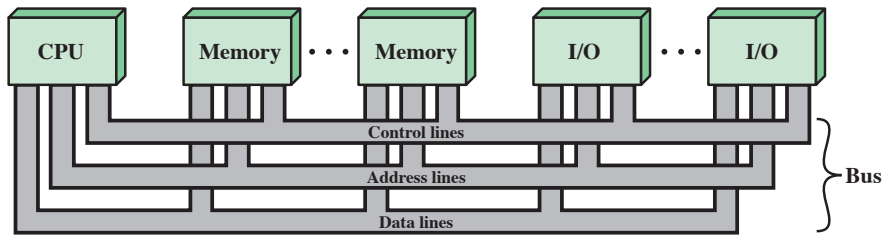
So, basically everything should be connected with everything.

- ▶ How do we make connections?
- ▶ Different technologies exist.
- ▶ In which topology should components be connected?
- ▶ What are the typical communication patterns?
- ▶ What is the relative importance and speed of components?

- Instruction Cycle
- Interrupts
- I/O Functionality
- Interconnection Structures
 - Bus Interconnection
 - QPI
 - PCIe

Bus

- ▶ Essentially a set of parallel wires, each called a **line**.
- ▶ Each line usually has a specific purpose: control, data, address.
- ▶ Components are then attached to the bus.
- ▶ Key characteristic: it's a shared transmission medium.
- ▶ Signals from one device are visible to all devices attached to the bus.
Two senders at the same time: garbage
- ▶ Was the dominant method for decades, but now only common in embedded systems.



Bus Lines

A bus may have hundreds of lines, for example:

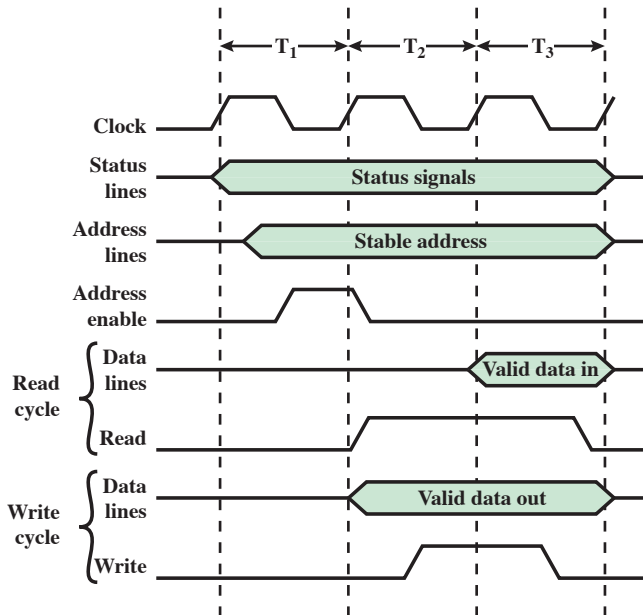
- ▶ 64 lines for specifying an address to read/write.
- ▶ 32, 64, or 128 lines for specifying data to write / read data.

This width is a key factor in the overall performance.

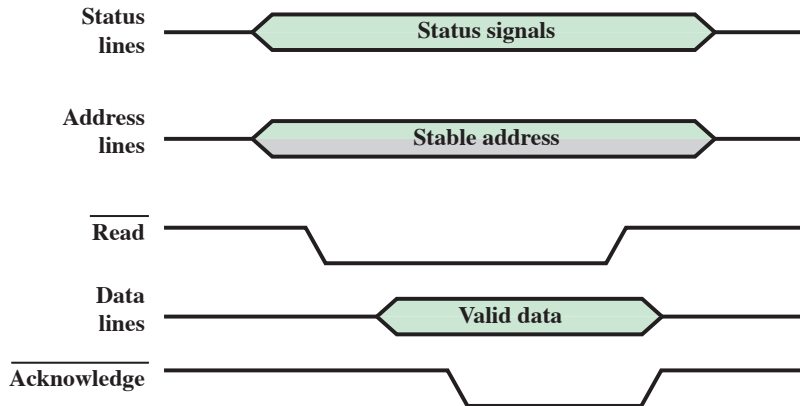
- ▶ Control lines
 - ▶ Clock signal (for a synchronous bus)
 - ▶ Operation signals (read or write, memory or I/O)
 - ▶ Transfer ACK, data has been read or placed on the bus as requested.
 - ▶ Bus request/grant, for coordination of who may use it at the moment.
Different arbitration schemes exist.
 - ▶ Interrupt request/grant

Lines can also be multiplexed, e.g., address and data through the same lines. Additional control lines indicate what they mean at a given time.

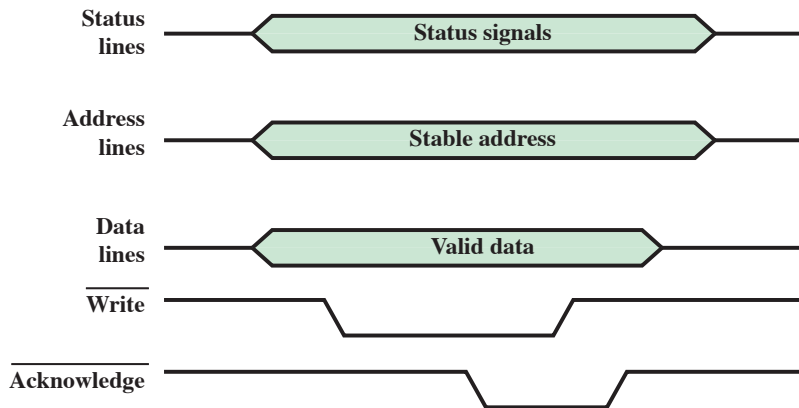
Synchronous Bus Operation



Asynchronous Bus Operation, Read

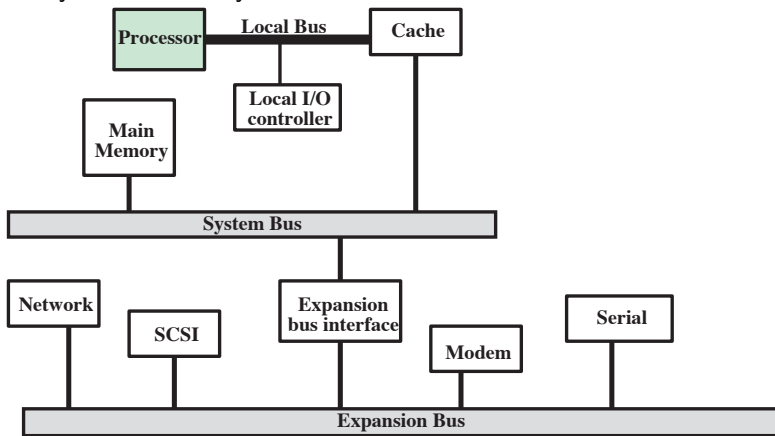


Asynchronous Bus Operation, Write



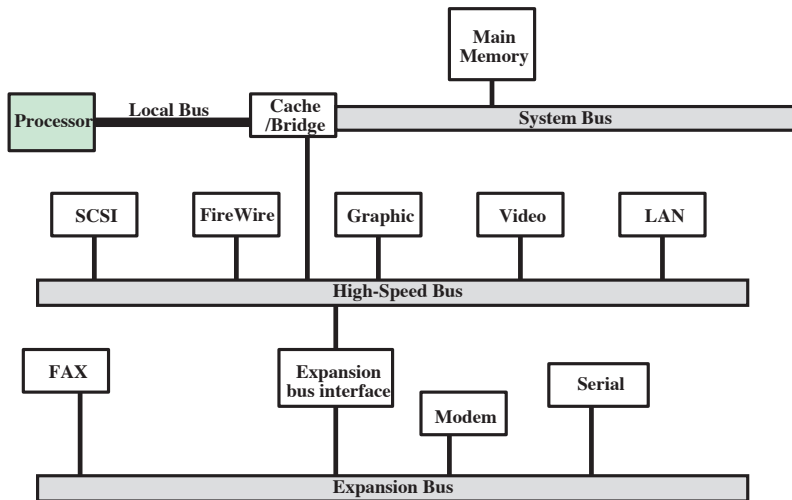
Bus Hierarchies

- ▶ **System bus:** a bus that connects major components (CPU, memory, I/O).
- ▶ Computers may have several busses for various levels of the system hierarchy.



(a) Traditional Bus Architecture

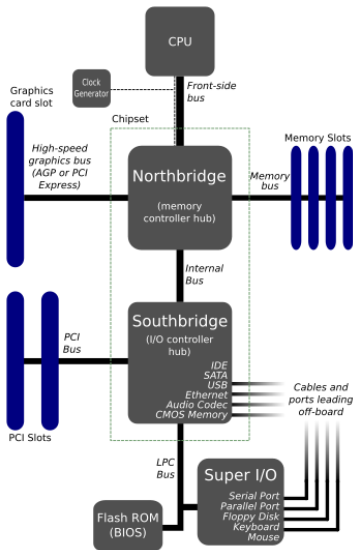
Need for Speed: Mezzanine Hierarchy



(b) High-Performance Architecture

Typical Bus-Based Organisation

Not commonly used since 2010.



[[https://en.wikipedia.org/wiki/Northbridge_\(computing\)](https://en.wikipedia.org/wiki/Northbridge_(computing))]

- Instruction Cycle
- Interrupts
- I/O Functionality
- Interconnection Structures
 - Bus Interconnection
 - QPI
 - PCIe

Point-to-Point Interconnect

Bus problems:

- ▶ To gain higher bandwidth busses were made wider and the clock frequency was increased.
- ▶ The higher frequencies make it increasingly difficult to perform arbitration and synchronization fast enough.
- ▶ An increase in number of CPU cores and amount of memory per chip only made the situation worse.

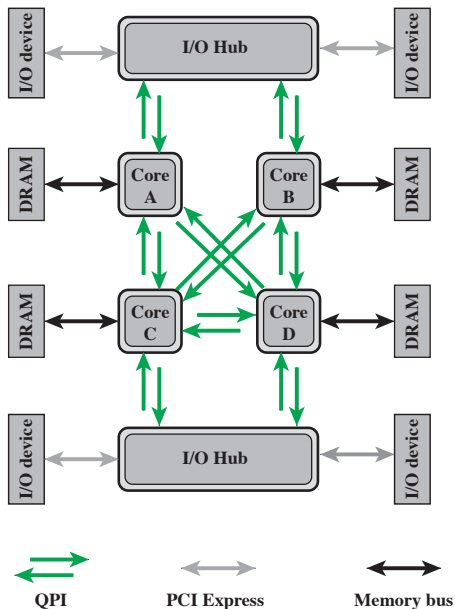
Current solution:

- ▶ No shared medium, only point-to-point interconnect.
- ▶ Different technology than single lines.
- ▶ Gives lower latency, higher data rate, and better scalability.

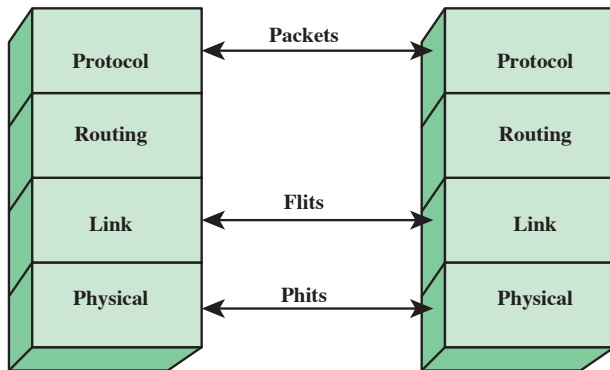
Representative Example: QuickPath Interconnect (QPI)

- ▶ Introduced by Intel in 2008 as replacement for the Front-side bus (FSB).
- ▶ AMD had a similar technology, HyperTransport, since 2001.
- ▶ Superseded by UltraPath Interconnect (UPI) in Skylake servers (2011).
- ▶ Primarily used for connecting processors, e.g., in servers with multiple CPUs.
- ▶ But has also been used internally in CPUs.
- ▶ Layered protocol architecture
 - ▶ Similar to how TCP/IP networks works.
 - ▶ In contrast to simple control signals in busses.
- ▶ Packetized data transfer
 - ▶ In contrast to a bit stream.
 - ▶ A sequence of packets, each with control headers and error control codes.

Imagined Example: Multicore Configuration Using QPI

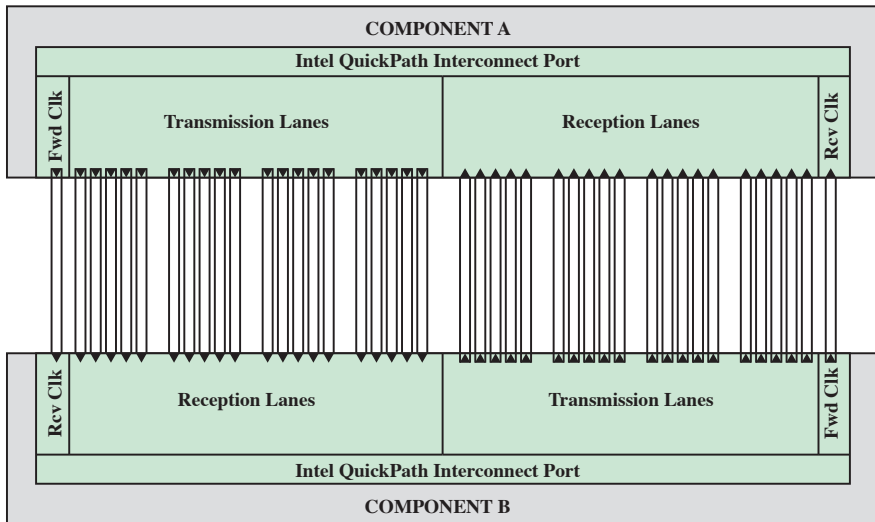


QPI Layers



- ▶ **Phit**: physical unit, 20 bits
- ▶ **Flit**: flow control unit, 80 bits

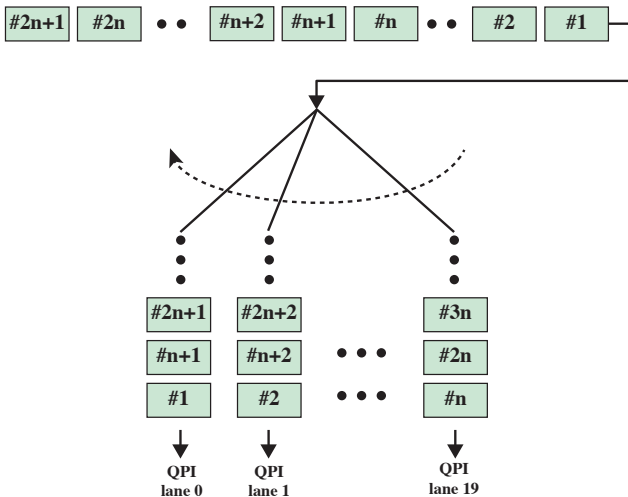
Physical Interface



Physical Layer

- ▶ 84 individual links/"wires"
 - ▶ Two wires is a unidirectional **lane**.
 - ▶ 20 data lanes in each direction.
 - ▶ 1 clock lane in each direction.
- ▶ Maximum speed is 6.4 GT/s (T: transfer) per lane.
 - ▶ 20 data lanes per direction gives 16 GiB/s per direction
- ▶ A lane transmits using **low-voltage differential signalling** (LVDS)
 - ▶ Also called **balanced transmission**.
 - ▶ The two wires form a loop with a resistor on the receiving side.
 - ▶ Current is send in one direction or the other to indicate 0 or 1.

Multilane Distribution



Bits in a flit are transmitted in a round-robin fashion on the physical lanes.

Link Layer

- ▶ Performs flow control and error control.
- ▶ Operates on 80 bit flits: 72 bit for message, 8 bits for error control.
- ▶ Error control:
 - ▶ Uses [cyclic redundancy check](#) (CRC).
 - ▶ Isolates higher layers from bit errors.
 - ▶ Requires buffering in the sender to allow resending.
- ▶ Flow control uses a credit based system:
 - ▶ Initialize with a number of credits.
 - ▶ Sending a flit costs a credit.
 - ▶ The receiver gives the credit back later.

Routing and Protocol Layer

Routing:

- ▶ QPI is point-to-point, so a packet may need to be send through multiple links.
- ▶ This layer is responsible for routing packets.
- ▶ Operates on 72-bit packages: 64 bit message, 8 bit header.
- ▶ Routing tables are defined by the firmware.
- ▶ For larger systems the tables can be complex and dynamic.
 - ▶ Which devices are populated (e.g., which CPU sockets)?
 - ▶ How are system resources partitioned?
 - ▶ Have some links failed currently?

Protocol:

- ▶ A packet is typically data send to/from a cache.
- ▶ This layer participates in the [cache coherency protocol](#).
Makes sure that values in caches and memory are consistent.
(More on this in a later lecture)

- Instruction Cycle
- Interrupts
- I/O Functionality
- Interconnection Structures
 - Bus Interconnection
 - QPI
 - PCIe

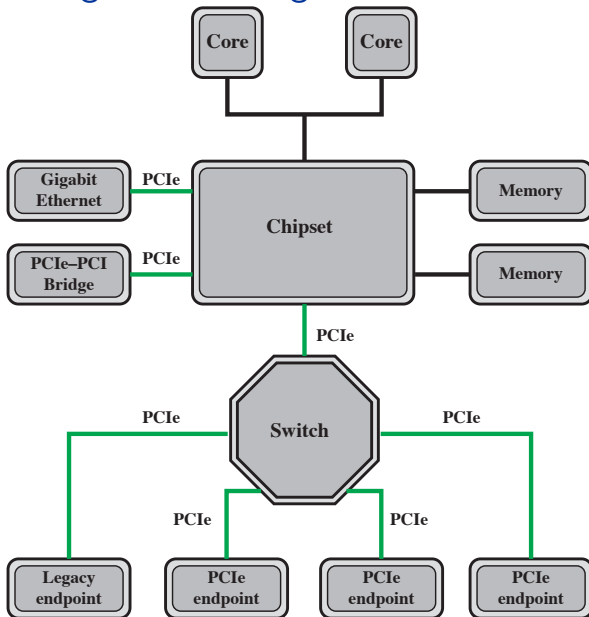
Peripheral Component Interconnect (PCI)

- ▶ First developed by Intel starting in 1990.
- ▶ Then made public domain and now developed in the PCI Special Interest Group.
- ▶ A processor-independent bus.
- ▶ Was widely adopted.
- ▶ Better performance for high-speed I/O subsystems than other bus specifications at the time.

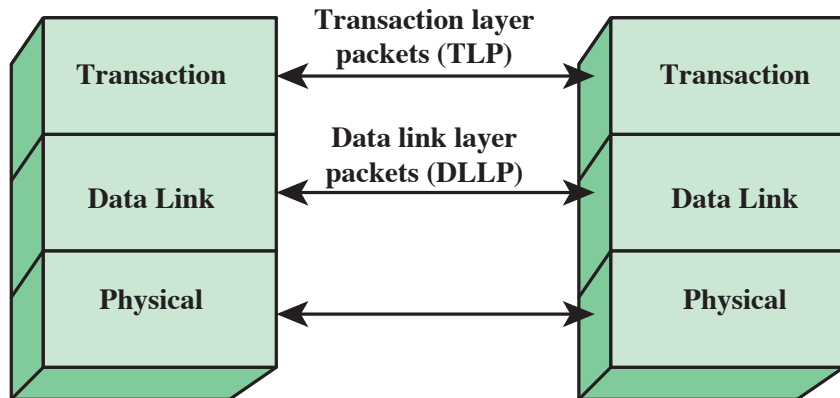
PCI Express (PCIe):

- ▶ Point-to-point interconnect scheme to replace the bus-based PCI.
- ▶ Has high data rate to support, e.g., Gigabit Ethernet.
- ▶ Deals with time dependent data streams and prioritization.

Abstract Configuration Using PCIe



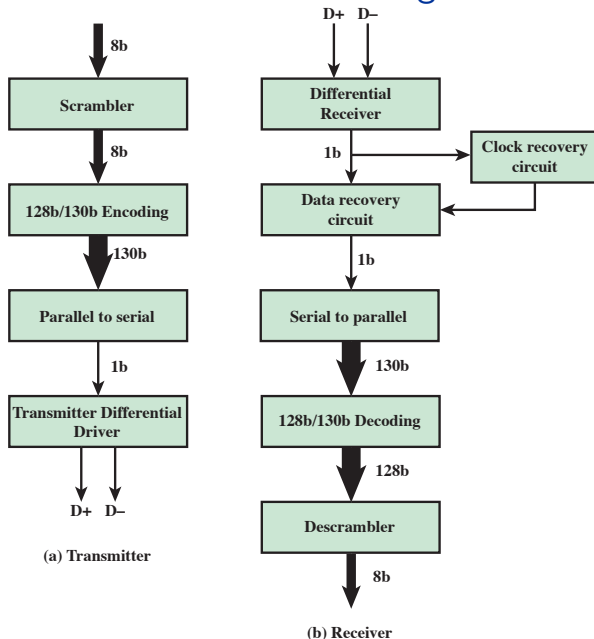
Also a Layered Protocol



Physical Layer

- ▶ 1, 2, 4, 8, or 16 bidirectional lanes per port.
- ▶ Uses differential signalling, like QPI.
- ▶ Uses multilane distribution, also like QPI.
- ▶ Unlike QPI, there is no clock signal.
- ▶ Instead, the clock is detected from transitions.
- ▶ Scrambling is used to limit runs of 0s and 1s.
- ▶ v3.0 to v5.0:
 - ▶ Each lane buffers 128 bits, and encodes them in a 130 bit block, to further aid clock synchronization.
 - ▶ The two extra bits 01 or 10, are also used to communicate what kind of data is transmitted.
- ▶ Since v6.0:
 - ▶ Uses PAM-4 signalling (4 voltage levels instead of 2).
 - ▶ Each lane buffers 242 bytes, and encodes them in a 256 byte block.
 - ▶ The extra data includes error correction.

PCIe Transmit and Receive Block Diagram



Transaction Layer

- ▶ Receives read/write requests from above, and creates packages for transmission via the data link layer below.
- ▶ Most transactions use a split transaction technique:
 - ▶ A request packet is sent out.
 - ▶ When the operation completes, the receiver sends back a completion packet.
 - ▶ Other packets can be transmitted in-between.
- ▶ Some transactions are posted transactions, i.e., without a response.
- ▶ TLP format supports 32-bit memory addressing and extended 64-bit memory addressing.
- ▶ Packets can have specific properties, e.g., “no-snoop”, “relaxed ordering”, “priority”.

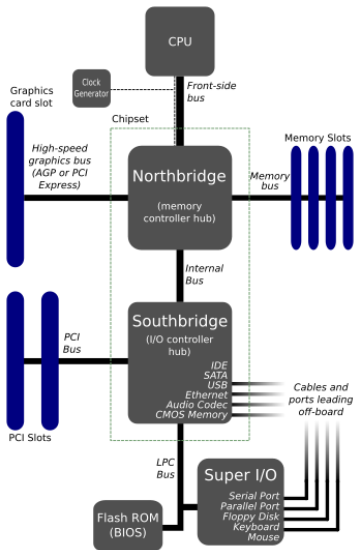
Transaction Layer and Address Spaces

Four different types of addresses, i.e., **spaces**, are supported:

- ▶ **Memory**: main memory and PCIe I/O devices.
- ▶ **I/O**: legacy PCI devices with reserved address ranges.
- ▶ **Configuration**: addresses to read/write configuration registers in I/O devices.
- ▶ **Message**: for control signals related to interrupts, error handling, and power management.

Typical Bus-Based Organisation

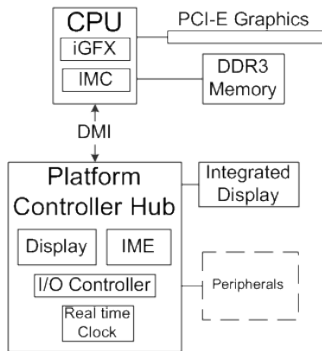
Not commonly used since 2010.



[[https://en.wikipedia.org/wiki/Northbridge_\(computing\)](https://en.wikipedia.org/wiki/Northbridge_(computing))]

Closer to Modern Organization (Intel)

The shrinking transistor size made it possible to integrate the northbridge in the CPU from ~2010.



Further shrinking has made it possible to integrate some of the southbridge/PCH as well.

This is called a **system-on-chip** (SoC) design.

[https://en.wikipedia.org/wiki/Platform_Controller_Hub]